SAÉ 1.05
Traiter des données (Chiffrement)



DEADLINE ===> 29 Janvier
Ce serait bien si on pouvait finir le projet pour ce weekend, histoire de rattraper ce qui n'a pas pu être fait pendant les vacances, et de pas trop speedrun le travail la semaine pro.

Nom	Prenom
Abdou	Imane
Baret	Alix
Houmadi	Soirihi

TP 1 : Configuration de gocryptfs avec pCloud et montage automatique au démarrage

Partie 1 : Configuration de gocryptfs avec redémarrage en cas d'échec.

Introduction :

Ce TP vous guidera à travers les étapes essentielles pour configurer gocryptfs, un outil de chiffrement performant, afin de sécuriser un répertoire spécifique.
Vous apprendrez à automatiser le montage de ce répertoire chiffré au démarrage de votre système, tout en garantissant que le client pCloud est opérationnel avant de procéder au montage.
Ce processus vous permettra non seulement de protéger vos données sensibles, mais aussi de vous familiariser avec l'intégration de solutions de stockage cloud sécurisées dans un environnement Linux.

Objectifs du TP :

- Installer gocryptfs et configurer un répertoire chiffré.
- Créer un fichier de mot de passe sécurisé pour gocryptfs.
- Configurer systemd pour monter automatiquement le répertoire chiffré au démarrage.
- Vérifier et dépanner la configuration.
- Répondre à des questions pour tester la compréhension.

Prérequis :

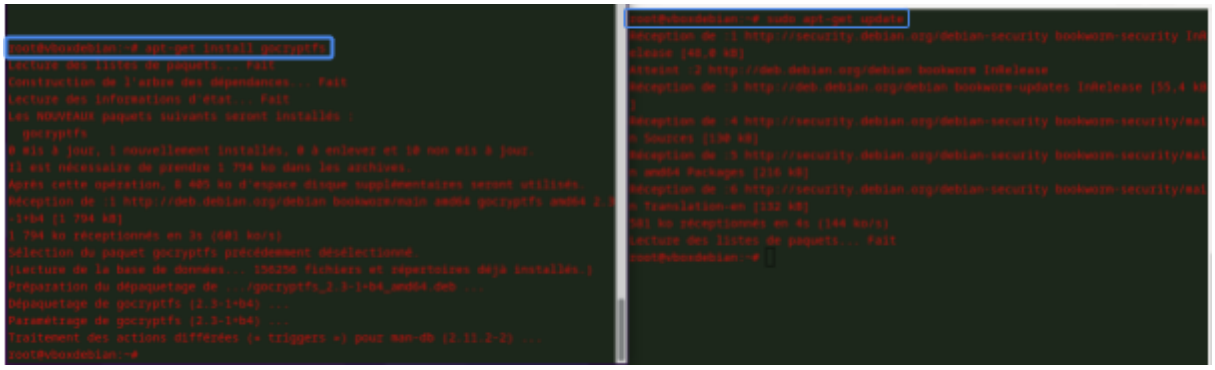
- **Système Debian**
- **Compte et client pCloud**
- **Connaissance des commandes Linux**
- **Données à chiffrer**

Étape 1 : Installation de gocryptfs

Premièrement, nous passons en super utilisateur pour mettre à jour les paquets puis installer gocryptfs avec les commandes :

#apt-get update

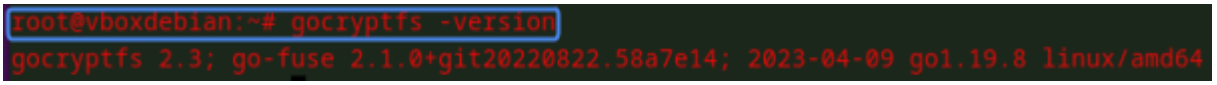
apt-get install gocryptfs



Nous avons mis à jour les paquets *(à droite)* puis nous avons installé gocryptfs*(à gauche)*.
Ces commandes ont dû être réalisées en 'root' puisqu'elle nécessite des droits d'administrateurs (installation et mise à jour des paquets oblige).

Question 1 : Quelle est la commande pour vérifier si gocryptfs est correctement installé ?

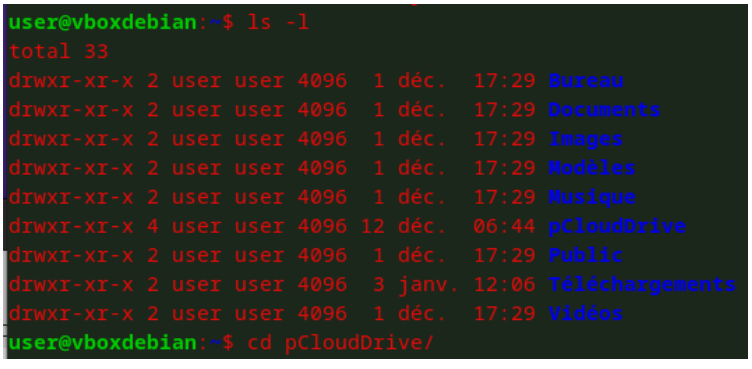
L'une des commande pour vérifier si gocryptfs est installé 'gocryptfs -version'



Si la commande ne renvoie pas d'erreur, c'est que gocryptfs est correctement installé.

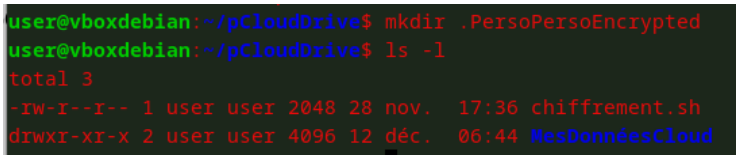
Étape 2 : Configuration initiale de gocryptf

1. Créez un répertoire pour contenir les fichiers chiffrés dans votre répertoire pCloud :



Affichage de l'arborescence et déplacement dans le dossier pCloudDrive

Création du dossier '.PersoPersoEncrypted' dans le dossier pCloudDrive



2. Initialisez le répertoire chiffré avec la commande :

'gocryptfs -init .PersoPersoEncrypted'

```
user@vboxdebian:~$ cd pCloudDrive/ && gocryptfs -init .PersoPersoEncrypted
Choose a password for protecting your files.
Password:
Repeat:

Your master key is:

    ffed9191-bb60f2dd-d2e8eb57-b0c75e81-
    979947c6-f6e9bde5-8c301401-3b111acc

If the gocryptfs.conf file becomes corrupted or you ever forget your password,
there is only one hope for recovery: The master key. Print it to a piece of
paper and store it in a drawer. This message is only printed once.
The gocryptfs filesystem has been created successfully.
You can now mount it using: gocryptfs .PersoPersoEncrypted MOUNTPOINT
user@vboxdebian:~/pCloudDrive$
```

Cette commande permet d'initier le dossier chiffré en lui installant les fichiers de configuration requis

```
user@vboxdebian:~/pCloudDrive$ ls -l .PersoPersoEncrypted/
total 1
-rw-r--r-- 1 user user 383  3 janv. 16:32 gocryptfs.conf
-rw-r--r-- 1 user user  16  3 janv. 16:32 gocryptfs.diriv
```

3. Montez le répertoire chiffré sur le répertoire cible :

J'ai créé un répertoire, l'ai affiché, puis j'ai monté mon dossier chiffré sur mon répertoire.

```
user@vboxdebian:~$ mkdir Perso
user@vboxdebian:~$ ls -l ~ | grep " Perso$"
drwxr-xr-x 2 user user 4096  3 janv. 16:44 Perso
user@vboxdebian:~$ gocryptfs pCloudDrive/.PersoPersoEncrypted/ Perso
Password:
Decrypting master key
Filesystem mounted and ready.
```

Question 2 : Quelle est la commande pour démonter un répertoire chiffré monté avec gocryptfs ?

C'est la commande `fusermount -u [path-to-mounted-directory]`
Dans notre cas, ce serait ' `fusermount -u ~/Perso` '

Étape 3 : Créer un fichier de mot de passe

1. Créez un fichier contenant le mot de passe pour gocryptfs et assurez-vous qu'il est sécurisé :

```
[user@machina]$ echo "votre_mot_de_passe" > /home/user/.gocryptfs_password
[user@machina]$ chmod 600 /home/user/.gocryptfs_password
```

`echo "motdepasse" > file` envoie l'entrée dans un fichier texte nommé '.gocryptfs_password'

```
user@vboxdebian:~$ echo "/:oipmlkjuyh" > /home/user/.gocryptfs_password
user@vboxdebian:~$ ls -la | grep ".gocryptfs *"
-rw-r--r--  1 user user   13  5 janv. 11:32 .gocryptfs_password
user@vboxdebian:~$ chmod 600 /home/user/.gocryptfs_password && ls -la .gocryptfs_password
-rw------- 1 user user 13  5 janv. 11:32 .gocryptfs_password
```

Question 3 : Pourquoi est-il important de sécuriser le fichier de mot de passe avec la commande chmod 600 ?

Celà permet que le fichier ne soit lisible (et écrivable) que par l'utilisateur qui a créé le fichier. Ainsi, le mot de passe est sécurisé et ne peut être obtenu

Étape 4 : Créer le fichier d'unité systemd

1. Créez ou modifiez le fichier d'unité systemd pour gérer le montage de gocryptfs :

```
[root@machina]# nano /etc/systemd/system/gocryptfs-perso.service

[Unit]
Description=Mount gocryptfs filesystem for PersoPerso
After=network.target home.mount
Wants=home.mount

[Service]
Type=idle
ExecStartPre=/bin/sleep 20
ExecStart=/usr/bin/gocryptfs --extpass "cat /home/user/.gocryptfs_password" /
home/user/pCloudDrive/.PersoPersoEncrypted /home/user/PersoPerso
ExecStop=/usr/bin/fusermount3 -u /home/user/PersoPerso
User=user
Group=user
StandardInput=tty-force
StandardOutput=journal
RemainAfterExit=true
Restart=on-failure
RestartSec=5s

[Install]
WantedBy=default.target
```

Ce fichier de service configure le déchiffrement automatique des données avec gocryptfs au démarrage.

Pour des questions de simplicité, mon dossier ne s'appelle pas 'PersoPerso' mais 'Perso'

```
root@vboxdebian:/# nano /etc/systemd/system/gocryptfs-perso.service
```

```
  GNU nano 7.2                                  /etc/systemd/system/gocryptfs-perso.service *
[Unit]
Description=Mount gocryptfs filesystem for Perso
After=network.target home.mount
Wants=home.mount

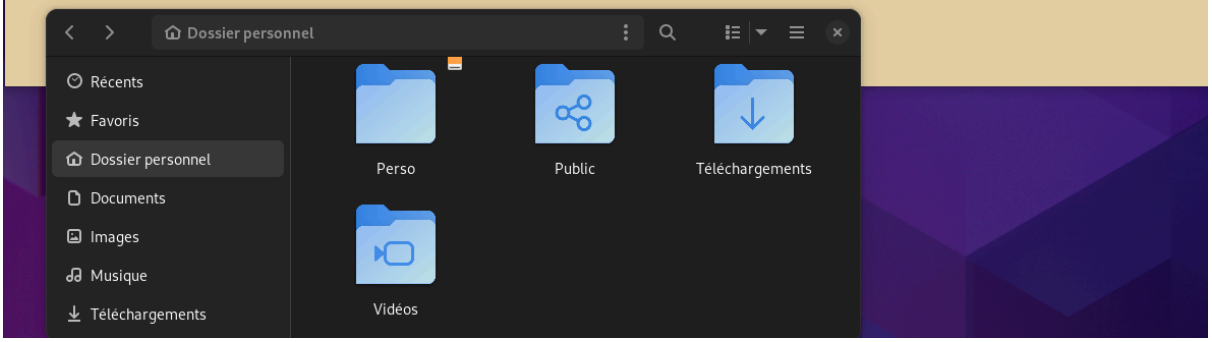
[Service]
Type=idle
ExeStartPre=/bin/sleep 20
ExecStart=/usr/bin/gocryptfs --extpass "cat /home/user/.gocryptfs_password" /home/user/pCloudDrive/.PersoPersoEncrypted /home/user/Perso
ExecStop=/usr/bin/fusermount3 -y /home/user/Perso
User=user
Group=user
StandartInput=tty-force
StandartOutput=journal
RemainAfterExit=true
Restart=on-failure
RestartSec=5s

[Install]
WantedBy=default.target
```

2. Rechargez les unités systemd et activez le service :

```
[root@machina]# systemctl daemon-reload
[root@machina]# systemctl enable gocryptfs-perso.service
[root@machina]# systemctl start gocryptfs-perso.service
```

```
root@vboxdebian:/# nano /etc/systemd/system/gocryptfs-perso.service
root@vboxdebian:/# systemctl daemon-reload
root@vboxdebian:/# systemctl enable gocryptfs-perso.service
Created symlink /etc/systemd/system/default.target.wants/gocryptfs-perso.service → /etc/systemd/system/gocryptfs-perso.service.
root@vboxdebian:/# systemctl start gocryptfs-perso.service
root@vboxdebian:/#
```



Question 4 : Que fait l'instruction ExecStartPre=/bin/sleep 20 dans le fichier d'unité systemd ?

Elle permet que lors de l'exécution du fichier d'unité systemd, celui-ci va rester dormant pendant 20 secondes avant de lancer le reste du script.

Étape 5 : Vérifier l'état du service

1. Vérifiez que le service fonctionne correctement :

```
[user@machina]$ systemctl status gocryptfs-perso.service
```

```
● gocryptfs-perso.service - Mount gocryptfs filesystem for Perso
     Loaded: loaded (/etc/systemd/system/gocryptfs-perso.service; enabled; preset: enabled)
     Active: active (exited) since Sun 2025-01-05 11:53:51 CET; 3h 31min ago
    Process: 7628 ExecStart=/usr/bin/gocryptfs --extpass cat /home/user/.gocryptfs_password /home/user/pCloudDrive/.PersoPersoEncrypted /home/user/Perso (code=exited, status=0>
   Main PID: 7628 (code=exited, status=0/SUCCESS)
      Tasks: 16 (limit: 2280)
     Memory: 15.8M
        CPU: 356ms
     CGroup: /system.slice/gocryptfs-perso.service
             ├─7634 /usr/bin/gocryptfs -fg -notifypid=7628 --extpass "cat /home/user/.gocryptfs_password" /home/user/pCloudDrive/.PersoPersoEncrypted /home/user/Perso
             └─7646 logger -t gocryptfs-7634-logger

janv. 05 11:53:51 vboxdebian systemd[1]: Started gocryptfs-perso.service - Mount gocryptfs filesystem for Perso.
janv. 05 11:53:51 vboxdebian gocryptfs[7634]: Reading password from extpass program "cat", arguments: ["/home/user/.gocryptfs_password"]
janv. 05 11:53:51 vboxdebian gocryptfs[7634]: Decrypting master key
janv. 05 11:53:52 vboxdebian gocryptfs[7634]: Filesystem mounted and ready.
```

Le fichier unité s'execute sans erreur apparente.
Il est chargé et actif, s'est executé comme il le devait en utilisant le fichier contenant le mot de passe, créé à l'étape 3.

Question 5 : Comment vérifier les journaux de systemd pour dépanner un service ?
La commande "systemctl status gocryptfs-perso.service" permets de vérifier les journaux d'un service pour le dépanner

Étape 6 : Redémarrer le système

1. Redémarrez votre système pour vérifier que le montage automatique fonctionne :

```
[root@machina]# reboot
```

```
user@vboxdebian:~$ su
Mot de passe :
root@vboxdebian:/home/user# source .bashrc
root@vboxdebian:/home/user# reboot
bash: reboot : commande introuvable
root@vboxdebian:/home/user# export PATH=$PATH:/sbin:/usr/sbin
root@vboxdebian:/home/user# reboot
```

2. Après le redémarrage, vérifiez que le répertoire est monté correctement : Perso est monté

Question 6 : Que se passe-t-il si gocryptfs essaie de monter le répertoire chiffré avant que pCloud ne soit actif ?

Gocryptfs ne pourra pas monter le répertoire chiffré si celui-ci n'existe pas/n'est pas trouvé dans le système de fichier de l'ordinateur, ainsi, le script ne fonctionnera pas, et gocryptfs ne fera rien.

Partie 2 : Configuration de gocryptfs avec continuation du démarrage en cas d'échec

Étape 1 : Création d'un script pour demander le mot de passe

1. Créez un script pour demander le mot de passe et monter le répertoire chiffré :

```
[root@machina]# nano /usr/local/sbin/mount_gocryptfs.sh

#!/bin/bash

# Demander le mot de passe
echo "Entrez le mot de passe pour gocryptfs:"
read -s PASSWORD

# Monter le répertoire chiffré
echo $PASSWORD | /usr/bin/gocryptfs --extpass "cat" /home/user/
pCloudDrive/.PersoPersoEncrypted /home/user/PersoPerso
```



Rendez le script exécutable :

```
[root@machina]# chmod +x /usr/local/sbin/mount_gocryptfs.sh
```

```
root@vboxdebian:/# chmod +x /usr/local/sbin/mount_gocryptfs.sh
root@vboxdebian:/# ls -l /usr/local/sbin/mount_gocryptfs.sh
-rwxr-xr-x 1 root root 250  6 janv. 17:47 /usr/local/sbin/mount_gocryptfs.sh
```

Étape 2 : Créer le fichier d'unité systemd pour la demande de mot de passe

1. Créez ou modifiez le fichier d'unité systemd pour exécuter le script au démarrage :

```
[root@machina]# nano /etc/systemd/system/gocryptfs-perso.service
```

```
[Unit]
Description=Mount gocryptfs filesystem for PersoPerso
After=network.target home.mount
Wants=home.mount

[Service]
Type=ic
ExecSta
User=us
Group=u
Standar
Standar
RemainA
Restart
Restart
StartLi
StartLi

[Instal
WantedB
```

```
 GNU nano 7.2          /etc/systemd/system/gocryptfs-perso.service
[Unit]
Description=Mount gocryptfs filesystem for Perso
After=network.target home.mount
Wants=home.mount

[Service]
Type=idle
ExecStart=/usr/local/sbin/mount_gocryptfs.sh
User=user
Group=user
StandartInput=tty-force
StandartOutput=journal
RemainAfterExit=true
Restart=on-failure
RestartSec=5s
StartLimitInterval=0
StartLimitBurst=3

[Install]
WantedBy=default.target
```

2. Rechargez les unités systemd et activez le service :

```
[root@machina]# systemctl daemon-reload
[root@machina]# systemctl enable gocryptfs-perso.service
[root@machina]# systemctl start gocryptfs-perso.service
```

```
root@vboxdebian:/# systemctl daemon-reload
root@vboxdebian:/# systemctl enable gocryptfs-perso.service
root@vboxdebian:/# systemctl start gocryptfs-perso.service
```

Question 7 : Que font les directives Restart=on-failure et RestartSec=5s dans le fichier d'unité systemd ?

Les directives 'Restart=on-failure' et 'RestartSec=5s' dans le fichier d'unité permette de :

- Relancer le script s'il y a une erreur
- Le relancer, après avoir attendu 5 secondes

Étape 3 : Redémarrer le système

1. Redémarrez votre système pour vérifier que le montage automatique fonctionne même si le mot de passe est incorrect :

```
[root@machina]# reboot
```

2. Après le redémarrage, vérifiez que le système continue de démarrer même si le mot de passe est incorrect :

Le système démarre bien, sans soucis

Question 8 : Comment configurer le fichier d'unité systemd pour limiter le nombre de tentatives de redémarrage ?

L'option "`StartLimitBurst`" définit le nombre maximal de tentatives de redémarrage autorisées dans un intervalle de temps spécifié.

Conclusion :

En suivant ces étapes, nous avons configuré gocryptfs pour monter automatiquement un répertoire chiffré au démarrage du système Debian, avec deux approches différentes :

L'une avec

redémarrage en cas d'échec et l'autre permettant au système de continuer à démarrer même si le mot de passe est incorrect.

Chapitre : Vérifications et dépannage

1 - Analyse des logs du système

Les logs du système sont une ressource essentielle pour vérifier le bon fonctionnement des services et diagnostiquer des problèmes. Sous Debian, vous pouvez consulter les journaux spécifiques à certains services ou événements en temps réel à l'aide de la commande '**tail -f**'

Par exemple, pour surveiller les journaux généraux du système :

```
[root@machina]# tail -f /var/log/syslog
```

Pour des événements liés au chiffrement et au montage de partitions, consultez les journaux du kernel :

```
[root@machina]# tail -f /var/log/kern.log
```

Cependant, il semble que le fichier 'syslog' ait été remplacé par un autre fichier, puisque celui-ci n'existe pas dans les fichiers de logs. De même pour le fichier 'kern.log'. Qui sont tout deux dépréciés, ce qui signifie que ce ne sont plus des fichiers utilisés par les derniers systèmes Debian.

Si vous utilisez un gestionnaire de démarrage ou des services comme systemd, combinez cette commande avec un filtre pour observer des événements spécifiques en temps réel :

```
[root@machina]# journalctl -f
```

Question 1 : Pourquoi est-il utile d'observer les logs en temps réel lors de l'exécution de services ou de commandes critiques ? Donnez un exemple concret.

Observer les logs en temps réel permet de détecter immédiatement des erreurs ou des comportements anormaux lors de l'exécution de services ou de commandes critiques. Cela facilite le diagnostic rapide et l'application de corrections sans attendre qu'un problème se manifeste pleinement. Par exemple, lors du montage automatique d'une partition chiffrée avec LUKS, surveiller les logs avec '`journalctl -f`' permet de voir instantanément si le

passe est accepté, si le périphérique est reconnu ou si une erreur de configuration empêche le montage. Cela évite un redémarrage inutile et permet d'intervenir rapidement en cas d'échec.

2 - Vérification des services et montages

Assurez-vous que les services ou montages sont actifs et fonctionnent comme prévu. Par exemple, utilisez la commande lsblk pour vérifier les partitions montées :

```
[root@machina]# lsblk
```

La commande `lsblk` permet d'afficher la liste des périphériques de stockage et leurs partitions, ainsi que leurs points de montage. Elle est utile pour vérifier si une partition est bien détectée et montée sur le système de fichiers.

```
root@VboxDebian:/home/utilisateur# lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   20G  0 disk
├─sda1   8:1    0  953M  0 part /boot
├─sda2   8:2    0    1K  0 part
├─sda3   8:3    0  9,3G  0 part /
├─sda4   8:4    0  7,9G  0 part /home
└─sda5   8:5    0  1,9G  0 part [SWAP]
sr0     11:0    1 1024M  0 rom
```

Cependant, elle n'est pas idéale pour un montage avec **gocryptfs**, car ce dernier utilise un système de fichiers basé sur **FUSE (Filesystem in Userspace)** plutôt qu'une partition classique. Comme `lsblk` ne répertorie que les périphériques de stockage physiques et leurs partitions, il ne permet pas de voir les répertoires montés via **FUSE**.

Vous pouvez également utiliser df pour afficher les systèmes de fichiers montés, leurs points de montage, et l'espace disque disponible :

```
[root@machina]# df -h
```

```
utilisateur@VboxDebian:~$ df -h
Sys. de fichiers Taille Utilisé Dispo Uti% Monté sur
udev                952M       0  952M   0% /dev
tmpfs               197M   1016K  196M   1% /run
/dev/sda3           9,1G    7,9G  758M  92% /
tmpfs               984M    512K  983M   1% /dev/shm
tmpfs               5,0M       0  5,0M   0% /run/lock
/dev/sda1           920M    138M  719M  17% /boot
/dev/sda4           7,7G    1,2G  6,2G  16% /home
tmpfs               197M     60K  197M   1% /run/user/1000
pCloud.fs           5,0G    119M  4,9G   3% /home/utilisateur/pCloudDrive
```

Même soucis avec la commande 'df -h', qui n'affiche que les systèmes de fichiers montés en tant que volumes classiques, pas ceux montés grâce à **FUSE** (comme Gocryptfs).

Enfin, pour confirmer que le répertoire gocryptfs est bien monté, vous pouvez exécuter :

Cependant, la commande 'mount | grep gocryptfs' permet d'afficher les périphériques montés, et se concentre sur l'affichage de la ligne contenant le terme 'gocryptfs'.

```
utilisateur@VboxDebian:~$ mount | grep gocryptfs
/home/utilisateur/pCloudDrive/.encryptedfiles on /home/utilisateur/Documents/MonMontage type fuse.gocryptfs (rw,nosuid,nodev,relatime,user_id=1000,group_id=1000,max_read=131072)
```

Ainsi, on peux observer que le montage de Gocryptfs est bien fonctionnel.

Question 2 : Expliquez pourquoi il est important de vérifier régulièrement l'état des montages lors de la configuration de services liés au stockage

Si les services de stockage et de chiffrement sont mal montés/configurés, ceux-ci ne seront pas fonctionnels et ne permettront pas le chiffrement, donc les données ne seront pas protégées.

TP2 : Créer et chiffrer une partition sous Debian

Partie 1 : Préparation de la partition

Commencez par identifier le disque où créer la partition. Utilisez la commande suivante pour afficher les partitions disponibles :
pour cela nous allons utiliser la commandes `~# $LSBK`

```
root@vboxdebian:~# lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   20G  0 disk
├─sda1   8:1    0  487M  0 part /boot
├─sda2   8:2    0 11,2G  0 part /
├─sda3   8:3    0  3,7G  0 part [SWAP]
├─sda4   8:4    0    1K  0 part
└─sda5   8:5    0  4,6G  0 part
sr0     11:0    1 1024M  0 rom
```

Une fois le disque choisi, créez une nouvelle partition sur ce disque en utilisant fdisk. Remplacez /dev/sdX par le nom de votre disque :

```
root@vboxdebian:~# fdisk -l
Disque /dev/sda : 20 GiB, 21474836480 octets, 41943040 secteurs
Modèle de disque : VBOX HARDDISK
Unités : secteur de 1 × 512 = 512 octets
Taille de secteur (logique / physique) : 512 octets / 512 octets
taille d'E/S (minimale / optimale) : 512 octets / 512 octets
Type d'étiquette de disque : dos
Identifiant de disque : 0x2d2861f3

Périphérique Amorçage    Début      Fin Secteurs Taille Id Type
/dev/sda1    *            2048   999423   997376   487M 83 Linux
/dev/sda2                999424 24436735 23437312  11,2G 83 Linux
/dev/sda3              34129920 41940991  7811072   3,7G 82 partition d'échange L
/dev/sda4              24438782 34129919  9691138   4,6G  5 Étendue
/dev/sda5              24438784 34129919  9691136   4,6G 83 Linux

Les entrées de la table de partitions ne sont pas dans l'ordre du disque.
```

Question 1 : Quelles informations le retour de la commande lsblk vous donne-t-il sur les disques et partitions ?
La commande `~# lsblk` affiche la liste des disques et partitions, leur taille, leur type, et leur point de montage si elles sont utilisées.

Partie 2 : Initialisation du chiffrement avec LUKS

Afin d'Installer le paquet cryptsetup ont va d'abord passer en Super utilisateur avec la commande `~# su -` puis ensuite on va réaliser les commande :

`'apt update'` et `'apt install cryptsetup'`

```
root@debian:/home/etudiant# apt update
Ign :1 http://deb.debian.org/debian bookworm InRelease
Ign :2 http://security.debian.org/debian-security bookworm-security InRelease
Ign :3 https://packagecloud.io/cs50/repo/debian bookworm InRelease
0% [Connexion à deb.debian.org] [Connexion à security.debian.org] [Connexion à packagecloud.io]
```

```
root@vboxdebian:~# apt install cryptsetup
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les paquets supplémentaires suivants seront installés :
```

Ensuite, nous initialiser le chiffrement LUKS sur la partition en remplaçant /dev/sdX1 par le nom de notre partition : /dev/sda5 (qui corresponds à la partition chiffré)

```
root@vboxdebian:~# cryptsetup luksFormat /dev/sda5
ATTENTION: Le périphérique /dev/sda5 contient déjà une signature pour un superblock « crypto_LUKS ».

ATTENTION !
==========
Cette action écrasera définitivement les données sur /dev/sda5.

Êtes-vous sûr ? (Typez « yes » en majuscules) : YES
```

La partition a été initialisée correctement, ses données ont donc été écrasées.

Nous devons maintenant déverouiller la partition en utilisant son nom d'identifiant, içi '`crypted_partition`' :

```
root@hostname:/home/user# cryptsetup open /dev/sda5 crypted_partition
Enter passphrase for /dev/sda5:
```

On entre le mot de passe de la partition, puis celle-ci est prête à recevoir les données à chiffrer.

Question 2 : Expliquez pourquoi il est important de ne pas oublier la phrase de passe LUKS et quelles pourraient être les conséquences d'un oubli.

Il est important de ne pas oublier la phrase de passe LUKS car, si vous l'oubliez, vous ne pourrez plus accéder à vos données chiffrées. Cela peut entraîner la perte de toutes vos données, car il n'y a pas d'autre méthode pour récupérer ces données sans la phrase de passe.

Partie 3 : Formatage de la partition chiffrée

Une fois la partition chiffrée déverrouillée, nous formatons celle-ci en ‘ext4’ pour y stocker les données

```
root@hostname:/home/user# mkfs.ext4 /dev/mapper/crypted_partition
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 1088256 4k blocks and 272544 inodes
Filesystem UUID: f5c7b19a-2a0a-45bd-a9bf-f04a2f199998
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done
```

Question 3 : Pourquoi est-il nécessaire de formater la partition après l’avoir chiffrée ?

Il est nécessaire de formater la partition après l'avoir chiffrée pour créer un système de fichiers (ici en ext4), ce qui permet au système d'exploitation de gérer et d'organiser les données de manière structurée.

Le chiffrement configure uniquement la partition pour protéger ses données, mais sans formatage, elle ne peut pas être utilisée pour stocker des fichiers.

```
root@hostname:/home/user# fdisk /dev/sda4
bash: fdisk: command not found
root@hostname:/home/user# dpkg -l | grep until-linux
root@hostname:/home/user# apt install util-linux
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
util-linux is already the newest version (2.38.1-5+deb12u2).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
root@hostname:/home/user# echo $PATH
/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games
root@hostname:/home/user# export PATH=$PATH:/sbin:/usr/sbin
root@hostname:/home/user# fdisk /dev/
Display all 166 possibilities? (y or n)
root@hostname:/home/user# fdisk /dev/sda4

Welcome to fdisk (util-linux 2.38.1).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS (MBR) disklabel with disk identifier 0xf075d03b.

Command (m for help):
```

La commande fdisk ne fonctionnait pas sur ma distribution de Linux, après vérification il me semble qu'elle était bien installé mais la variable $PATH n'y permettait pas l'accès
J'ai donc ajouté les emplacements des dossiers 'sbin' (Superuser Binaries) à la variable $PATH grâce à la commande 'export'

```
Commande (m pour l'aide) : F

Espace non partitionné /dev/sda4 : 7,89 GiB, 8472494080 octets, 16547840 secteurs
Unités : secteur de 1 × 512 = 512 octets
Taille de secteur (logique / physique) : 512 octets / 512 octets

Début      Fin Secteurs Taille
 2048 16549887 16547840   7,9G

Commande (m pour l'aide) : q

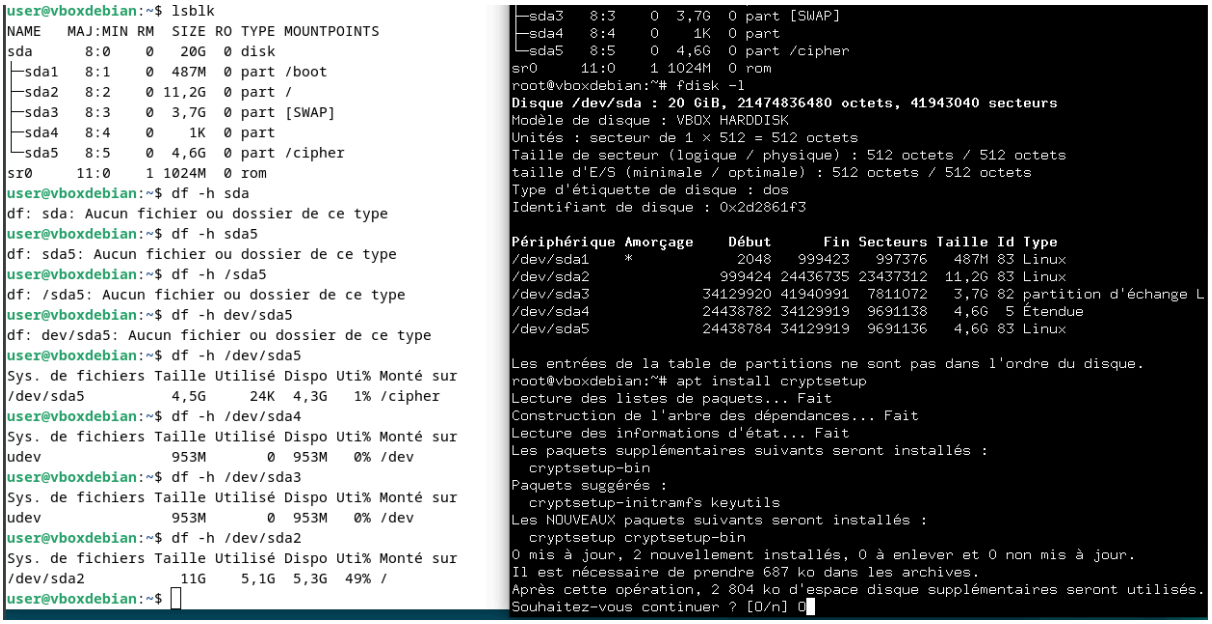
root@VboxDebian:~# lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   20G  0 disk
├─sda1   8:1    0  953M  0 part /boot
├─sda2   8:2    0    1K  0 part
├─sda3   8:3    0  9,3G  0 part /
├─sda4   8:4    0  7,9G  0 part /home
└─sda5   8:5    0  1,9G  0 part [SWAP]
sr0     11:0    1 1024M  0 rom
root@VboxDebian:~# apt install cryptsetup
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les paquets suivants ont été installés automatiquement et ne sont plus nécessaires :
  libfreerdp-server2-2 libfuse3-3 libmutter-11-0 libntfs-3g89 libtss2-tctildr0 mutter-common
Veuillez utiliser « apt autoremove » pour les supprimer.
Les paquets supplémentaires suivants seront installés :
  cryptsetup-bin
Paquets suggérés :
  cryptsetup-initramfs keyutils
Les NOUVEAUX paquets suivants seront installés :
  cryptsetup cryptsetup-bin
0 mis à jour, 2 nouvellement installés, 0 à enlever et 0 non mis à jour.
Il est nécessaire de prendre 687 ko dans les archives.
Après cette opération, 2 804 ko d'espace disque supplémentaires seront utilisés.
Souhaitez-vous continuer ? [O/n] O
Réception de :1 http://depot-debian.univ-reunion.fr/debian bookworm/main amd64 cryptsetup-bin amd64 2:2.6.1-4~deb12u2 [474 kB]
13% [1 cryptsetup-bin 111 kB/474 kB 23%]
```

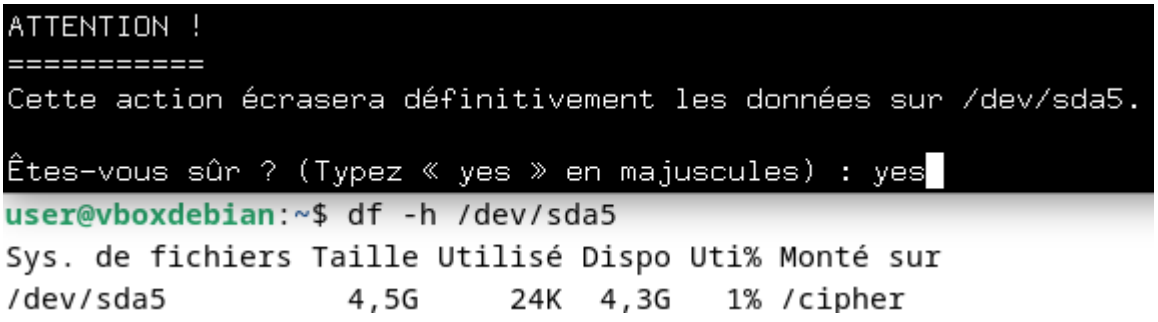
Installation de cryptsetup

```
root@VboxDebian:/# df -h /
Sys. de fichiers Taille Utilisé Dispo Uti% Monté sur
/dev/sda3          9,1G    6,4G  2,3G  75% /
root@VboxDebian:/# df -h /home
Sys. de fichiers Taille Utilisé Dispo Uti% Monté sur
/dev/sda4          7,7G    955M  6,4G  13% /home
root@VboxDebian:/# lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   20G  0 disk
├─sda1   8:1    0  953M  0 part /boot
├─sda2   8:2    0    1K  0 part
├─sda3   8:3    0  9,3G  0 part /
├─sda4   8:4    0  7,9G  0 part /home
└─sda5   8:5    0  1,9G  0 part [SWAP]
sr0     11:0    1 1024M  0 rom
root@VboxDebian:/# cp -a /home/* /homebackup
```

Je dois formater une partition pour pouvoir la chiffrer avec LUKS, j'ai donc dû réinstaller un debian où j'ai alloué une partition "/cipher" que j'allouerai au stockage/au chiffrement des fichiers.

J'installe ensuite cryptsetup pour pouvoir chiffrer mes données (on peux observer les partitions)





```
root@vboxdebian:~# lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   20G  0 disk
├─sda1   8:1    0  487M  0 part /boot
├─sda2   8:2    0 11,2G  0 part /
├─sda3   8:3    0  3,7G  0 part [SWAP]
├─sda4   8:4    0    1K  0 part
└─sda5   8:5    0  4,6G  0 part
sr0     11:0    1 1024M  0 rom
root@vboxdebian:~# cryptsetup open /dev/sda5 crypted_partition
Saisissez la phrase secrète pour /dev/sda5 :
root@vboxdebian:~# mkfs.ext4 /dev/mapper/crypted_partition
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 1207296 4k blocks and 301920 inodes
Filesystem UUID: aedb2ef5-8aaf-414f-b1c6-dcbcd8492c44
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

root@vboxdebian:~# lsblk
NAME                  MAJ:MIN RM  SIZE RO TYPE  MOUNTPOINTS
sda                     8:0    0   20G  0 disk
├─sda1                  8:1    0  487M  0 part  /boot
├─sda2                  8:2    0 11,2G  0 part  /
├─sda3                  8:3    0  3,7G  0 part  [SWAP]
├─sda4                  8:4    0    1K  0 part
└─sda5                  8:5    0  4,6G  0 part
  └─crypted_partition 254:0    0  4,6G  0 crypt
```

J’ai donc monté la partition chiffré sur ma partition sda5.

Partie 4 : Configuration pour le montage automatique au démarrage

Nous obtenons l'UUID de la partition chiffrée pour la configuration de /etc/crypttab et /etc/fstab avec la commande : `'blkid /dev/sdaX'` en remplaçant 'X' par le numéro de la partition.

```
root@vboxdebian:/home/user# nano /etc/crypttab
```

```
root@vboxdebian:/home/user# blkid /dev/sda4
/dev/sda5: UUID="3df0597a-3ce3-474e-913c-e5ea9ac0624d" TYPE="crypto_LUKS" PARTUUID="2d2861f3-05"
```

Puis, nous ajoutons l'UUID (obtenu grâce à la manipulation précédente) dans /etc/crypttab en remplaçant les valeurs adéquates : ici le UIID de la partition cible

```
  GNU nano 7.2                          /etc/crypttab *
# <target name> <source device>         <key file>      <options>
crypted_partition UUID=3df0597a-3ce3-474e-913c-e5ea9ac0624d none luks
```

Nous avons modifié /etc/crypttab, et nous devons maintenant modifier '`/etc/fstab`' pour configurer le montage automatique de la partition chiffré

```
root@vboxdebian:/home/user# nano /etc/fstab
```

```
  GNU nano 7.2                                                    /etc/fstab
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# systemd generates mount units based on this file, see systemd.mount(5).
# Please run 'systemctl daemon-reload' after making changes here.
#
# <file system> <mount point>   <type>  <options>       <dump>  <pass>
# / was on /dev/sda2 during installation
UUID=3b7b81bf-802b-4a41-b584-048fe3256662 /               ext4    errors=remount-ro 0       1
# /boot was on /dev/sda1 during installation
UUID=217c6a97-5523-448e-bdc9-4f0d70d137df /boot           ext4    defaults        0       2
# /cipher was on /dev/sda5 during installation

# swap was on /dev/sda3 during installation
UUID=c53ce293-36bd-4a73-bd69-5e56cebd0f6b none            swap    sw              0       0
/dev/sr0        /media/cdrom0   udf,iso9660 user,noauto     0       0

/dev/mapper/crypted_partition           /mnt/crypted      ext4    defaults        0       2
```

Nous ajoutons la ligne requise pour permettre le montage automatique de la partition chiffrée.

[redacted]

Question 4 : Quelle est l'utilité de l'UUID dans la configuration de /etc/crypttab et /etc/fstab ?

L'UUID (Universal Unique Identifier) est utilisé dans les fichiers de configuration `/etc/crypttab` et `/etc/fstab` pour identifier de manière unique une partition ou un système de fichiers. Son utilité réside dans la stabilité des identifiants, la configuration robuste qu'ils permettent, ainsi que la simplification des tâches administratives liés au système de fichier.

Partie 5 : Redémarrage et tests de la configuration

Redémarrez le système pour tester la configuration. Vous devriez être invité à entrer la phrase de passe pour déverrouiller la partition. Utilisez ensuite lsblk ou df -h pour confirmer que la partition est correctement montée :

```
[root@machina]# lsblk
```

```
[root@machina]# df -h
```

(on a mis un 'sleep 10' avant pour avoir le temps de prendre la capture d'écran)

```
root@vboxdebian:/home/user# sleep 10; reboot
```

Nous avons redémarré notre machine comme demandé, et celle-ci semble bien fonctionner.

```
/: clean, 203444/732960 files, 1650745/2929664 blocks
[    3.890426] systemd[1]: Invalid DMI field header.

Please enter passphrase for disk VBOX_HARDDISK (crypted_partition) on /mnt/crypted::**********_
```

Question 5 : Quelle commande utilisez-vous pour vérifier que la partition chiffrée est bien montée ? Justifiez votre choix.

Pour vérifier qu'une partition chiffrée est bien montée, la commande `lsblk` est idéale. Elle affiche une vue détaillée des périphériques de stockage, y compris les partitions et leurs points de montage. Cela permet de confirmer non seulement que la partition chiffrée a été déverrouillée (apparaissant dans `/dev/mapper`), mais aussi qu'elle est montée au bon emplacement dans le système de fichiers. Ce choix est justifié par la clarté des informations fournies, qui incluent l'arborescence des périphériques, les noms, les tailles et les points de montage, facilitant ainsi une vérification rapide et fiable.

Partie 6 : Test d'accès et démontage

Créez un fichier de test pour vérifier les permissions, puis démontez et verrouillez la partition :

```
root@vboxdebian:~# touch /mnt/crypted/testfile
root@vboxdebian:~# cd /mnt/crypted
root@vboxdebian:/mnt/crypted# tree
-bash: tree : commande introuvable
root@vboxdebian:/mnt/crypted# ls
lost+found  testfile
```

En 'root' je peux créer et modifier des fichiers dans cette partition

```
user@vboxdebian:~$ touch /mnt/crypted/testfile
touch: impossible de faire un touch '/mnt/crypted/testfile': Permission non accordée
user@vboxdebian:~$ ls -ld /mnt/crypted
drwxr-xr-x 3 root root 4096  2 déc.  14:03 /mnt/crypted
```

Mais celà ne fonctionne pas quand je suis en 'user', puisque je n'ai pas la permission d'accéder au dossier cible.

Je démonte la partition crypté, on voit içi qu'après la commande le point de montage /crypted_partition n'apparaît plus.

```
root@vboxdebian:/# umount /mnt/crypted
root@vboxdebian:/# lsblk
NAME                  MAJ:MIN RM  SIZE RO TYPE  MOUNTPOINTS
sda                     8:0    0   20G  0 disk
├─sda1                  8:1    0  487M  0 part  /boot
├─sda2                  8:2    0 11,2G  0 part  /
├─sda3                  8:3    0  3,7G  0 part  [SWAP]
├─sda4                  8:4    0    1K  0 part
└─sda5                  8:5    0  4,6G  0 part
  └─crypted_partition 254:0    0  4,6G  0 crypt
sr0                    11:0    1 1024M  0 rom
```

Question 6 : Que signifie verrouiller une partition avec cryptsetup close et pourquoi est-ce utile ?

Verrouiller une partition avec `cryptsetup close` signifie refermer l'accès à une partition chiffrée, en déconnectant son périphérique mappé dans `/dev/mapper`. Cela est essentiel pour protéger les données sensibles en empêchant tout accès non autorisé, même si un utilisateur a accès au système. Ce processus garantit que la partition reste sécurisée tant que la clé ou la phrase de passe n'est pas fournie pour la déverrouiller à nouveau. Cela est particulièrement utile dans les environnements partagés ou lorsqu'un système est laissé sans surveillance, car les données deviennent totalement inaccessibles sans les informations d'authentification nécessaires. En somme, cette action renforce la confidentialité et prévient les accès involontaires ou malveillants.

Conclusion et questions de synthèse :

Nous avons appris à chiffrer une partition sous Debian, à la configurer pour un montage automatique, et à vérifier que tout fonctionne correctement.

Question de synthèse 1 : Quelle est l'importance du chiffrement d'une partition pour la sécurité des données ?

Question de synthèse 2 : En cas de perte de la phrase de passe, que se passerait-il pour les données stockées sur la partition chiffrée ?

Chapitre : Vérifications et dépannage de LUKS

Analyse des logs du système

Les logs sont essentiels pour diagnostiquer les erreurs ou confirmer le bon fonctionnement des services liés à la partition chiffrée. Utilisez la commande tail -f pour observer les journaux en temps réel. Par exemple :

Surveillez les logs du système général :

```
[root@machina]# tail -f /var/log/syslog
```

Vérifiez les logs liés aux opérations de cryptsetup et de montage :

```
root@machina]# tail -f /var/log/kern.log
```

Comme déjà indiqué dans le premier TP, les fichiers ciblés n’existent pas, ou n’existe plus.

Seul la commande : permettrait d'obtenir un résultat répondant à nos attentes

```
[root@machina]# journalctl | grep cryptsetup
```

```
janv. 23 12:30:53 vboxdebian systemd[1]: Created slice system-systemd\x2dcryptsetup.slice - Cryptsetup Units Slice.
janv. 23 12:30:54 vboxdebian systemd[1]: Starting systemd-cryptsetup@crypted_partition.service - Cryptography Setup for crypted_partition...
janv. 23 12:32:11 vboxdebian systemd-cryptsetup[296]: Set cipher aes, mode xts-plain64, key size 512 bits for device /dev/disk/by-uuid/3df0597a-3ce3-474e-913c-e5ea9ac0624d.
janv. 23 12:32:15 vboxdebian systemd[1]: Finished systemd-cryptsetup@crypted_partition.service - Cryptography Setup for crypted_partition.
janv. 23 12:32:15 vboxdebian systemd[1]: Reached target cryptsetup.target - Local Encrypted Volumes.
janv. 23 12:46:54 vboxdebian systemd[1]: Stopped target cryptsetup.target - Local Encrypted Volumes.
janv. 23 12:46:54 vboxdebian systemd[1]: Stopping systemd-cryptsetup@crypted_partition.service - Cryptography Setup for crypted_partition...
janv. 23 12:46:54 vboxdebian systemd-cryptsetup[2786]: Device crypted_partition is not active.
janv. 23 12:46:54 vboxdebian systemd-cryptsetup[2786]: Volume crypted_partition already inactive.
janv. 23 12:46:54 vboxdebian systemd[1]: systemd-cryptsetup@crypted_partition.service: Deactivated successfully.
janv. 23 12:46:54 vboxdebian systemd[1]: Stopped systemd-cryptsetup@crypted_partition.service - Cryptography Setup for crypted_partition.
janv. 23 12:47:11 vboxdebian systemd[1]: Created slice system-systemd\x2dcryptsetup.slice - Cryptsetup Units Slice.
janv. 23 12:47:11 vboxdebian systemd[1]: Starting systemd-cryptsetup@crypted_partition.service - Cryptography Setup for crypted_partition...
janv. 28 10:57:17 vboxdebian systemd[1]: Created slice system-systemd\x2dcryptsetup.slice - Cryptsetup Units Slice.
janv. 28 10:57:18 vboxdebian systemd[1]: Starting systemd-cryptsetup@crypted_partition.service - Cryptography Setup for crypted_partition...
janv. 28 11:00:35 vboxdebian systemd-cryptsetup[299]: Set cipher aes, mode xts-plain64, key size 512 bits for device /dev/disk/by-uuid/3df0597a-3ce3-474e-913c-e5ea9ac0624d.
janv. 28 11:00:39 vboxdebian systemd[1]: Finished systemd-cryptsetup@crypted_partition.service - Cryptography Setup for crypted_partition.
janv. 28 11:00:39 vboxdebian systemd[1]: Reached target cryptsetup.target - Local Encrypted Volumes.
janv. 29 07:02:04 vboxdebian systemd[1]: Created slice system-systemd\x2dcryptsetup.slice - Cryptsetup Units Slice.
janv. 29 07:02:04 vboxdebian systemd[1]: Starting systemd-cryptsetup@crypted_partition.service - Cryptography Setup for crypted_partition...
janv. 29 07:05:11 vboxdebian systemd-cryptsetup[302]: Set cipher aes, mode xts-plain64, key size 512 bits for device /dev/disk/by-uuid/3df0597a-3ce3-474e-913c-e5ea9ac0624d.
janv. 29 07:05:14 vboxdebian systemd[1]: Finished systemd-cryptsetup@crypted_partition.service - Cryptography Setup for crypted_partition.
janv. 29 07:05:14 vboxdebian systemd[1]: Reached target cryptsetup.target - Local Encrypted Volumes.
root@vboxdebian:/home/user# journalctl | grep cryptsetup
```

Comme on le vois, la commande ci-dessus permets d'obtenir les données de 'journalctl' lié à 'cryptsetup'

Question 1 : Pourquoi est-il important d'examiner les logs lorsque le montage automatique d'une partition chiffrée échoue ?

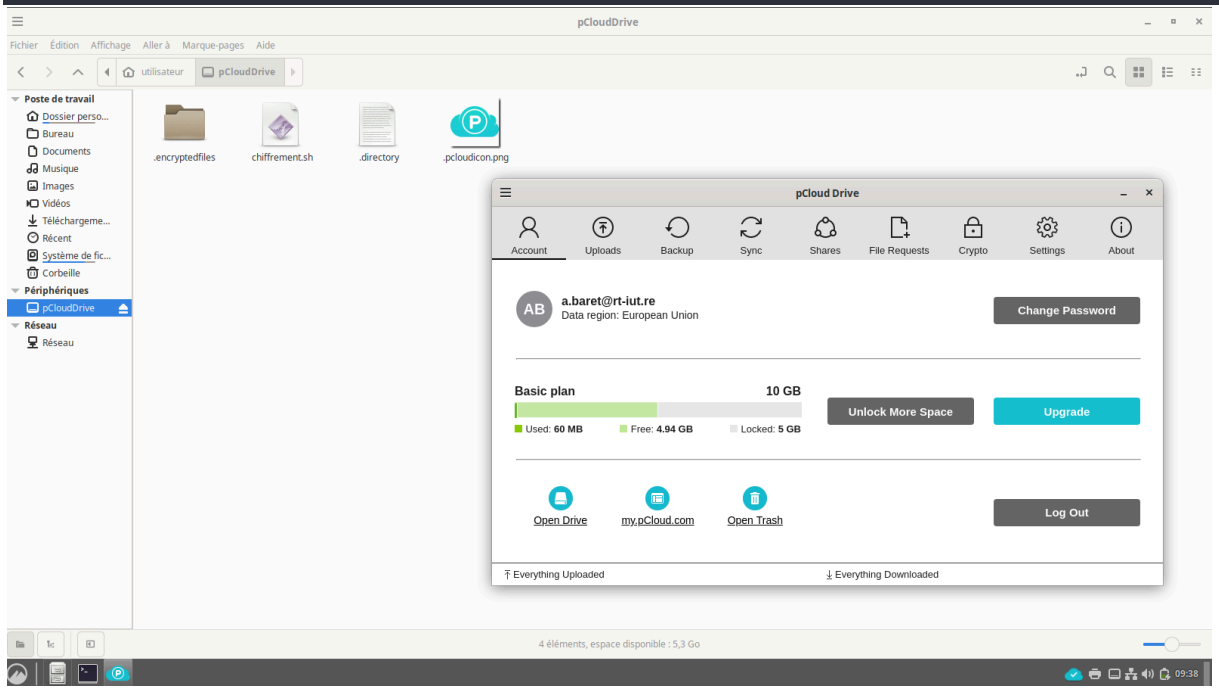
Tests fonctionnels de la partition chiffrée

Je ferme la

partition, elle n’apparait plus dans sda5

```
root@vboxdebian:/# cryptsetup close crypted_partition
root@vboxdebian:/# lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   20G  0 disk
├─sda1   8:1    0  487M  0 part /boot
├─sda2   8:2    0 11,2G  0 part /
├─sda3   8:3    0  3,7G  0 part [SWAP]
├─sda4   8:4    0    1K  0 part
└─sda5   8:5    0  4,6G  0 part
```

```
utilisateur@VboxDebian:~$ echo "/:oipmlkjuyh" > /home/utilisateur/.gocryptfs_password
utilisateur@VboxDebian:~$ chmod 600 /home/utilisateur/.gocryptfs_password
utilisateur@VboxDebian:~$ cat /home/utilisateur/
Téléchargements/    .viminfo           .Xauthority   .xsession-errors
.xsession-errors.old
.bash_history.bash_logout .bashrc      Bureau/               .cache/   .config/
.dmrc  Documents/
.face  .face.icon      .lesshst     .local/    .gocryptfs_password    Modèles/
Musique/    .pcloud/
pcloudDrive/    .profile    Public/     Images/      .mozilla/    .pki/     Vidéos/
utilisateur@VboxDebian:~$ cat /home/utilisateur/.gocryptfs_password
/:01pmlkjuyh
utilisateur@VboxDebian:~$ chown utilisateur /home/utilisateur/.gocryptfs_password
utilisateur@VboxDebian:~$ chown root /home/utilisateur/.gocryptfs_password
chown: modification du propriétaire de '/home/utilisateur/.gocryptfs_password':
Opération non permise utilisateur@VboxDebian:~$ ls -1
/home/utilisateur/.gocryptfs_password -IW------- 1 utilisateur utilisateur 13 3 déc.
18:27 /home/utilisateur/.gocryptfs_password
```



TP3 : Intégration des deux méthodes de chiffrement

Partie 1 : Préparation de la partition chiffrée avec LUKS

Commencez par créer une partition chiffrée en suivant les étapes du premier TP, utilisant *cryptsetup* et LUKS. Voici les commandes de base pour préparer et monter la partition chiffrée.

```
[root@machina]# lsblk
```

```
[root@machina]# fdisk /dev/sdX
```

```
[root@machina]# cryptsetup luksFormat /dev/sdX1
```

```
[root@machina]# cryptsetup open /dev/sdX1 crypted_partition
```

```
[root@machina]# mkfs.ext4 /dev/mapper/crypted_partition
```

```
[root@machina]# mount /dev/mapper/crypted_partition /mnt/crypted
```

Vérifiez que la partition est bien montée sur `/mnt/crypted`.

0. Mise à jours des paquets

```
root@hostname:/home/user# sudo apt update
Hit:1 http://depot-debian.univ-reunion.fr/debian bookworm InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
```

1. Affichage des partitions du système de fichier

```
root@hostname:/home/user# lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   20G  0 disk
├─sda1   8:1    0  7.9G  0 part /
├─sda2   8:2    0  3.7G  0 part [SWAP]
├─sda3   8:3    0  4.2G  0 part /home
└─sda4   8:4    0  4.2G  0 part
sr0     11:0    1 1024M  0 rom
```

2. Affichage des informations lié à la partition à chiffrer :

```
root@hostname:/home/user# fdisk -l /dev/sda4
Disk /dev/sda4: 4.17 GiB, 4474273792 bytes, 8738816 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xfd30647c

Device      Boot Start     End Sectors  Size Id Type
/dev/sda4p1       2048 8738815 8736768  4.2G  5 Extended
```

3. Initialisation de la partition chiffré avec ‘cryptsetup’ :

```
root@hostname:/home/user# cryptsetup luksFormat /dev/sda4
WARNING: Device /dev/sda4 already contains a 'dos' partition signature.

WARNING!
========
This will overwrite data on /dev/sda4 irrevocably.

Are you sure? (Type 'yes' in capital letters): YES
Enter passphrase for /dev/sda4:
Verify passphrase:
root@hostname:/home/user# lsblk
```

La partition ‘sda4’ a été initialisé pour qu’elle soit utilisée avec le chiffrement LUKS.

4. Ouverture de la partition chiffré avec ‘cryptsetup’ :

Cette commande ouvre une partition chiffrée configurée avec LUKS

```
root@hostname:/home/user# cryptsetup open /dev/sda4 crypted_partition
Enter passphrase for /dev/sda4:
root@hostname:/home/user# ?:oipmlkjuyh
```

5. Création du système de fichier EXT 4 sur la partition chiffré :

```
root@hostname:/home/user# mkfs.ext4 /dev/mapper/crypted_partition
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 1088256 4k blocks and 272544 inodes
Filesystem UUID: f5c7b19a-2a0a-45bd-a9bf-f04a2f199998
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done
```

Cette commande formate le dossier mappé cible, permettant de créer un système de fichier utilisant le format 'ext 4'.

6. Montage du système de fichier sur la partition chiffré :

Nous devons créer le point de montage puisqu'il n'existait pas.

```
root@hostname:/home/user# mkdir -pv /mnt/crypted
mkdir: created directory '/mnt/crypted'
root@hostname:/home/user# mount /dev/mapper/crypted_partition /mnt/crypted
```

Montage de la partition sur le dossier qui a été créé

Question 1 : Pourquoi utiliser un chiffrement sur la partition avant d'y ajouter un autre niveau de chiffrement par répertoire ?

L'utilisation d'un chiffrement sur la partition permet que celle-ci soit bien moins déchiffrable par une personne n'ayant pas le code.
Ainsi, tant que l'ordinateur n'a pas été allumé et dévérouillé, les fichiers resteront chiffrés.
Même si un utilisateur a réussi à déchiffrer

Partie 2 : Installation et préparation de gocryptfs

Installez le paquet *gocryptfs* si ce n'est pas déjà fait :

```
[root@machina]# apt update
```

```
[root@machina]# apt install gocryptfs
```

Créez ensuite un dossier dans la partition chiffrée pour y stocker le répertoire chiffré gocryptfs :

```
[root@machina]# mkdir /mnt/crypted/gocryptfs_data
```

```
[root@machina]# mkdir /mnt/crypted/gocryptfs_mount
```

1. Installation de gocryptfs

```
root@hostname:/home/user# apt-get install gocryptfs
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  gocryptfs
0 upgraded, 1 newly installed, 0 to remove and 90 not upgraded.
Need to get 1,794 kB of archives.
After this operation, 8,405 kB of additional disk space will be used.
Get:1 http://depot-debian.univ-reunion.fr/debian bookworm/main amd64 gocryptfs amd64 2.3-1+b4 [1,794 kB]
Fetched 1,794 kB in 4s (511 kB/s)
Selecting previously unselected package gocryptfs.
(Reading database ... 146886 files and directories currently installed.)
Preparing to unpack .../gocryptfs_2.3-1+b4_amd64.deb ...
Unpacking gocryptfs (2.3-1+b4) ...
Setting up gocryptfs (2.3-1+b4) ...
Processing triggers for man-db (2.11.2-2) ...
```

Nous utilisons le gestionnaire de paquet ***apt-get*** pour installer l'utilitaire ***gocryptfs***

2. Création des dossiers utilisés par Gocryptfs

```
root@hostname:/home/user# mkdir -v /mnt/crypted/gocryptfs_data /mnt/crypted/gocryptfs_mount
mkdir: created directory '/mnt/crypted/gocryptfs_data'
mkdir: created directory '/mnt/crypted/gocryptfs_mount'
```

Je créé les dossiers qui seront utilisés par Gocryptfs :

- Un dossier '***data***' qui contiendra les données chiffré
- Un dossier '***mount***' qui servira de dossier de montage

Partie 3 : Configuration du répertoire chiffré avec gocryptfs

Initialisez le répertoire chiffré gocryptfs à l'intérieur de la partition chiffrée :

```
[root@machina]# gocryptfs -init /mnt/crypted/gocryptfs_data
```

1. Initialisation de gocryptfs

```
root@hostname:/home/user# gocryptfs -init /mnt/crypted/gocryptfs_data/
Choose a password for protecting your files.
Password:
Repeat:

Your master key is:

    442092e7-35ddaff9-b6ba3194-6bed3c96-
    a3673e62-fc0c0934-9f489ccb-28e2627e

If the gocryptfs.conf file becomes corrupted or you ever forget your password,
there is only one hope for recovery: The master key. Print it to a piece of
paper and store it in a drawer. This message is only printed once.
The gocryptfs filesystem has been created successfully.
You can now mount it using: gocryptfs /mnt/crypted/gocryptfs_data MOUNTPOINT
root@hostname:/home/user#
```

On utilise la commande '***gocryptfs -init /mnt/crypted/gocryptfs_data***' pour initialiser le dossier qui contiendra les données chiffrés.

2. Montage du répertoire chiffré sur 'mnt/crypted/gocryptfs_data'

Montez le répertoire chiffré gocryptfs dans `/mnt/crypted/gocryptfs_mount` :

```
[root@machina]# gocryptfs /mnt/crypted/gocryptfs_data /mnt/crypted/
gocryptfs_mount
```

```
root@hostname:/home/user# gocryptfs /mnt/crypted/gocryptfs_data/ /mnt/crypted/gocryptfs_mount/
Password:
Decrypting master key
Filesystem mounted and ready.
```

On utilise l'utilitaire '***gocryptfs***' pour chiffrer les fichiers qui seront mis dans le dossier de montage (*gocryptfs_mount*), qui seront présenté, chiffré, dans le dossier '*gocryptfs_data*'.

Question 2 : Quel est l'intérêt d'ajouter un répertoire chiffré gocryptfs dans une partition déjà chiffrée par LUKS ?

L'intérêt d'un double chiffrement est d'offrir une sécurité optimale, puisque des données, plus elles sont chiffrés, plus elles seront impossible à déchiffrer (ou décrypter) par bruteforce ou d'autres techniques

Partie 4 : Tests et vérifications

1. Test du fonctionnement de gocryptfs

Testez le bon fonctionnement du répertoire chiffré gocryptfs en créant un fichier de test dans `/mnt/crypted/gocryptfs_mount` :

```
[root@machina]# touch /mnt/crypted/gocryptfs_mount/testfile
```

```
root@hostname:/home/user# touch /mnt/crypted/gocryptfs_mount/testfile
root@hostname:/home/user# cd /mnt/crypted/gocryptfs_mount/ && ls -l
total 0
-rw-r--r-- 1 root root 0 Jan 21 02:17 testfile
```

Pour tester le bon fonctionnement de mon montage avec gocryptfs, je créé un fichier dans le dossier de montage.

```
root@hostname:/mnt/crypted/gocryptfs_mount# cd ../gocryptfs_data/ ; ls -l
total 8
-r-------- 1 root root 383 Jan 21 01:23 gocryptfs.conf
-r--r--r-- 1 root root  16 Jan 21 01:23 gocryptfs.diriv
-rw-r--r-- 1 root root   0 Jan 21 02:17 yBFPx0pWV8D-m3DNwT641g
```

Comme on le voit sur cette capture, le dossier '*gocryptfs_data/*' contient un fichier (qui n'était pas là avant), dont le titre est chiffré, et qui représente le fichier 'testfile', chiffré.

Démontage de gocryptfs

Ensuite, démontez le répertoire gocryptfs et la partition chiffrée pour vous assurer que les données sont bien protégées :

```
[root@machina]# fusermount -u /mnt/crypted/gocryptfs_mount
```

```
[root@machina]# umount /mnt/crypted
```

Nous démontons le dossier de montage, puis la partition chiffré

```
root@hostname:~# fusermount -u /mnt/crypted/gocryptfs_mount
root@hostname:~# umount /mnt/crypted
umount: /mnt/crypted: target is busy.
root@hostname:~# umount /mnt/crypted
root@hostname:~# cd /mnt/crypted/
root@hostname:/mnt/crypted# ls
root@hostname:/mnt/crypted#
```

(un terminal était ouvert sur '/mnt/crypted', je l'ai fermé, puis umount a pu fonctionné)

Comme on peux le voir, le dossier /mnt/crypted ne contient rien (donc la partition chiffré a bien été démonte :)

Sources

Gocryptfs :

https://github.com/rfjakob/gocryptfs/blob/master/Documentation/MANPAGE.md - Page github de gocryptfs
https://opensource.com/article/19/8/how-encrypt-files-gocryptfs?ref=dailydev - Article expliquant l'utilisation

LaTeX:

https://www.freecodecamp.org/news/learn-latex-full-course/?ref=dailydev - LaTeX Course

Consigne : Pour moi, la SAE 1.05 nous demande de :

- Réaliser les TP (Gocryptfs,
- Produire des comptes Rendu de ceux-ci

- Et enfin, produire un guide répondant aux attentes de la SAE, soit :

Comment traiter des données ? Comment sécuriser des données ? Comment chiffrer des données ?

Expliquer comment utiliser l'utilitaire Gocryptfs - Comment chiffrer un répertoire spécifique ?

Expliquer comment chiffrer avec Cryptsetup - Comment chiffrer une partition ?

\title{Compte Rendu SAE - 1.05 - Traiter des données}
\author{Abdou Imane, Baret Alix, Houmadi Souhiri}

```
┌──(kali㉿kali)-[~/Documents]
└─$ tree
.
└── chiffrement
    └── mydoss
        ├── small.txt
        └── Test.txt

3 directories, 2 files

┌──(kali㉿kali)-[~/Documents]
└─$ cd chiffrement

┌──(kali㉿kali)-[~/Documents/chiffrement]
└─$ mkdir .cipher

┌──(kali㉿kali)-[~/Documents/chiffrement]
└─$ la
.cipher  mydoss

┌──(kali㉿kali)-[~/Documents/chiffrement]
└─$ gocryptfs -init .cipher
Choose a password for protecting your files.
Password:
Repeat:

Your master key is:

    fbe8557b-08afbc1e-7ff024f3-5a05b287-
    3a4daf11-e4001f06-dd91c4c8-b86bdc2e

If the gocryptfs.conf file becomes corrupted or you ever forget your password,
there is only one hope for recovery: The master key. Print it to a piece of
paper and store it in a drawer. This message is only printed once.
The gocryptfs filesystem has been created successfully.
You can now mount it using: gocryptfs .cipher MOUNTPOINT
```

Comme on peut le voir içi j'ai initialisé le dossier cible qui contiendra les documents chiffrés. (Ici sur Kali mais je l'ai déjà fait sous Debian facile et je le ferai sous ma nouvelle vm Debian)

```
┌──(kali㉿kali)-[~/Documents/chiffrement]
└─$ ll
total 4
drwxrwxr-x 2 kali kali 4096  9 nov.  19:52 mydoss

┌──(kali㉿kali)-[~/Documents/chiffrement]
└─$ la
.cipher  mydoss
```

Je monte ensuite mon dossier qui sera utilisé comme répertoire source soit le dossier contenant les fichiers à chiffrer.

kali@kali: ~/Documents/chiffrement/.cipher

Fichier Actions Éditer Vue Aide

```
└─$ gocryptfs .cipher befcipher
Password:
Decrypting master key
Filesystem mounted and ready.

┌──(kali㉿kali)-[~/Documents/chiffrement]
└─$ cp -rip mydoss befcipher

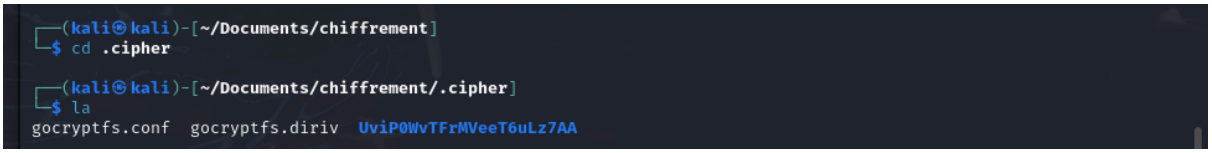
┌──(kali㉿kali)-[~/Documents/chiffrement]
└─$ la
befcipher  .cipher  mydoss

┌──(kali㉿kali)-[~/Documents/chiffrement]
└─$ tree
.
├── befcipher
│   └── mydoss
│       ├── small.txt
│       └── Test.txt
└── mydoss
    ├── small.txt
    └── Test.txt

4 directories, 4 files
```

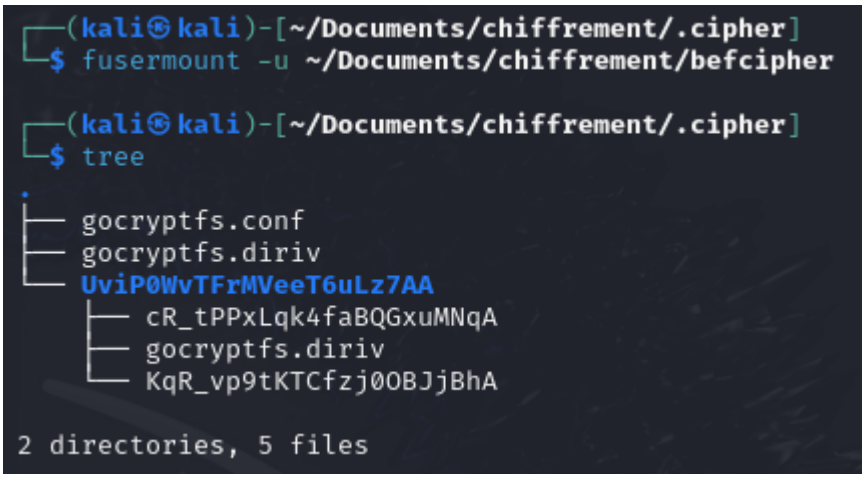
On peut

observer que les fichiers ont bien été copiés et collés dans le fichier de montage (qui n'apparaît pas via ‘tree’ puisque les répertoires commençant par un point ne sont pas affichés)



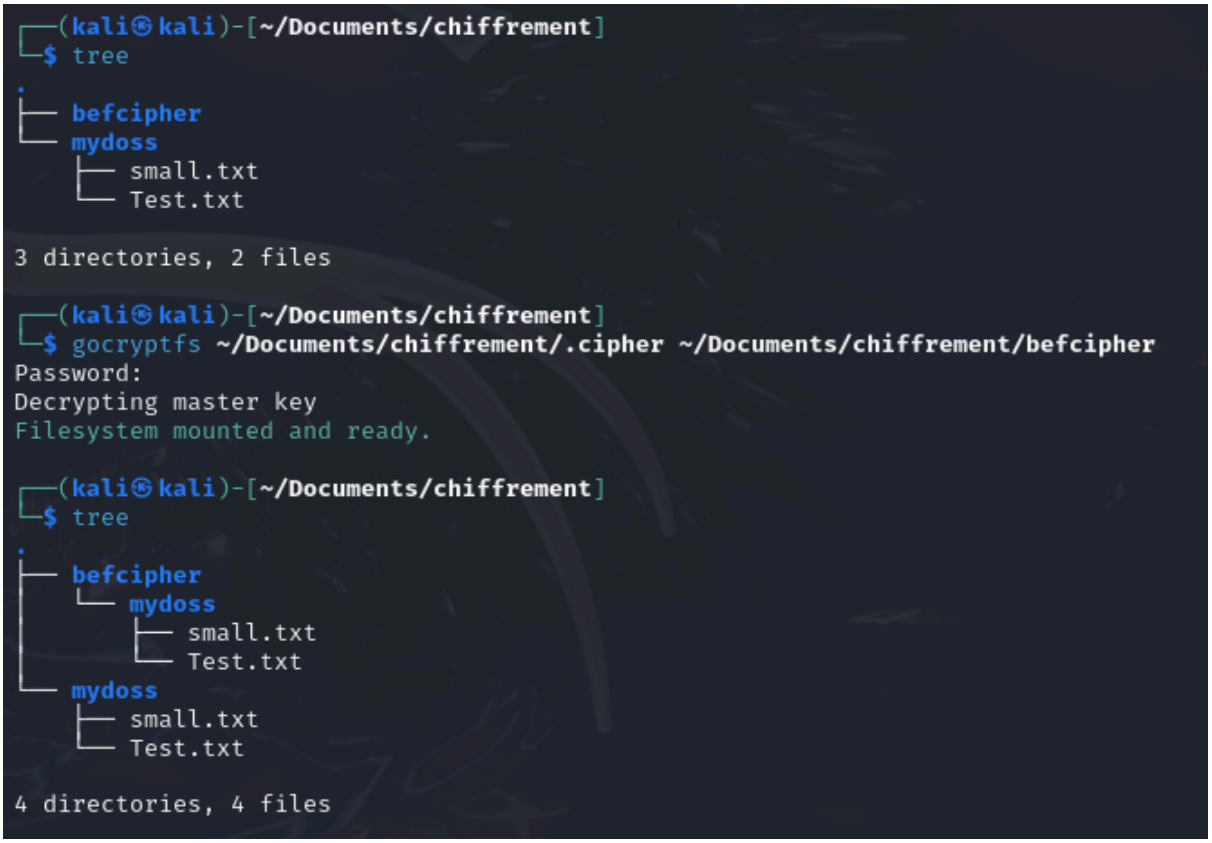
Pourtant les fichiers sont bel et bien présents, mais chiffrés, dans le répertoire cible.

C’est bien mais comment je sais que ça marche alors ? Et les fichiers vont-ils se chiffrer automatiquement sans demande préalable ?
Une fois le dossier monté, les commandes exécutées, la ‘machine mis en marche’, gocryptfs va automatiquement chiffrer toutes les données/les fichiers qui seront mises à sa disposition dans le point de montage (le répertoire source du dossier chiffré).
C’est plutôt pratique car une fois la manipulation effectuée, il est plutôt simple de chiffrer ses données (par exemple avant d’éteindre son ordinateur).



J’ai démonte le dossier contenant les données en clair, comme le montre la première commande.

Ainsi, les données chiffrées sont toujours présentes dans le dossier ‘.cipher’, et pourront être déchiffrées avec la clé lors du prochain ‘montage’ avec gocryptfs.



Ainsi, on peut déchiffrer les données et les chiffrer grâce aux outils proposés par l'utilitaire 'gocryptfs', comme on peut l'observer dans l'image au dessus.
J'ai juste eu à rentrer le mot de passe et j'ai obtenu mes données déchiffrées dans le dossier cible.

Je vais utiliser 'cp' pour copier les fichiers non chiffrés et les mettre dans mon dossier de montage.

C'est plus utile d'avoir 3 dossiers car le répertoire de montage doit être vide lorsque l'on fait la manip, ce qui est un peu embêtant.

Options courantes de `cp`

Voici un tableau des options les plus couramment utilisées avec `cp` et leur utilité :

Option	Description
`-r` ou `-R`	Copie récursive : permet de copier les dossiers et leur contenu, y compris les sous-dossiers.
`-i`	Interactif : demande confirmation avant de remplacer les fichiers existants dans le dossier de destination.
`-p`	Préserver les attributs : conserve les permissions, l'horodatage, et les autres attributs du fichier.
`-u`	Met à jour : ne copie le fichier que si celui du dossier source est plus récent que celui de destination, ou si ce dernier n'existe pas.
`-v`	Verbose : affiche chaque fichier copié. Utile pour suivre l'avancement de la copie.
`-a`	Archive : copie les fichiers et dossiers en conservant leurs attributs (équivaut à `-rp`), pour une copie exacte du dossier source.
`-n`	No-clobber : ne remplace pas les fichiers existants dans le dossier de destination.
`--backup`	Crée une sauvegarde des fichiers existants dans le dossier de destination avant de les remplacer.
`--parents`	Préserve l'arborescence complète : copie également la hiérarchie de dossiers jusqu'au fichier spécifié.

Maintenant, je vais créer un fichier de mot de passe sécurisé pour gocryptfs.

J'utilise la commande que nous avons vu en R108 pour génerer un mot de passe aléatoire (complexe, donc plus sécurisé pour le chiffrement des données car beaucoup plus dur à trouver par bruteforce/dictionnaire ou autre technique de décryptage).

Exemple :

```
┌──(kali㉿kali)-[~/Documents/chiffrement]
└─$ tr -dc A-Za-z0-9 < /dev/urandom | head -c44
GM5DN2nnVgXLrwZPVc4an68rrg3tCcIvuH7qXFfssJEC
```

Ainsi je peux chiffrer mes données avec un mot de passe très sécurisé (bien que ce n'est pas pratique car si je l'oublie, vu la difficulté qu'il présente, ça va être compliqué d'accéder à mes données (genre impossible)).

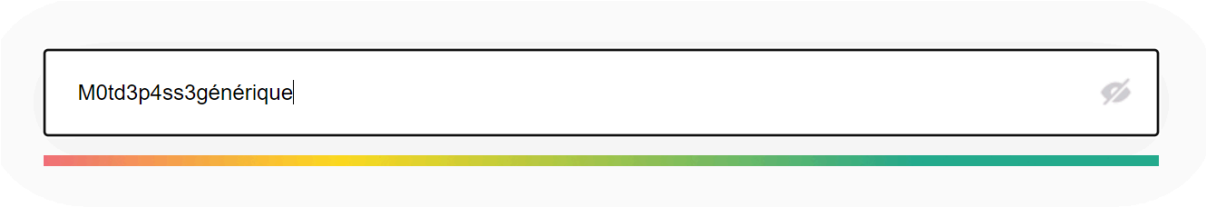
Alors je créer un mot de passe plus compliqué mais que je pourrais retenir puisqu'il est en rapport avec moi.

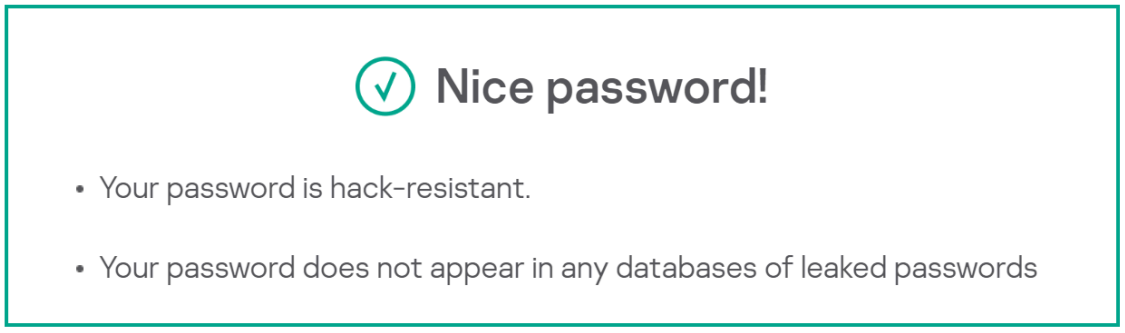
```
AlixBaretVaSeFaireExclure - Un mot de passe drôle et qui se retient
```

```
[user@machina]$ echo "votre_mot_de_passe" > /home/bruno/.gocryptfs_password
[user@machina]$ chmod 600 /home/bruno/.gocryptfs_password
```

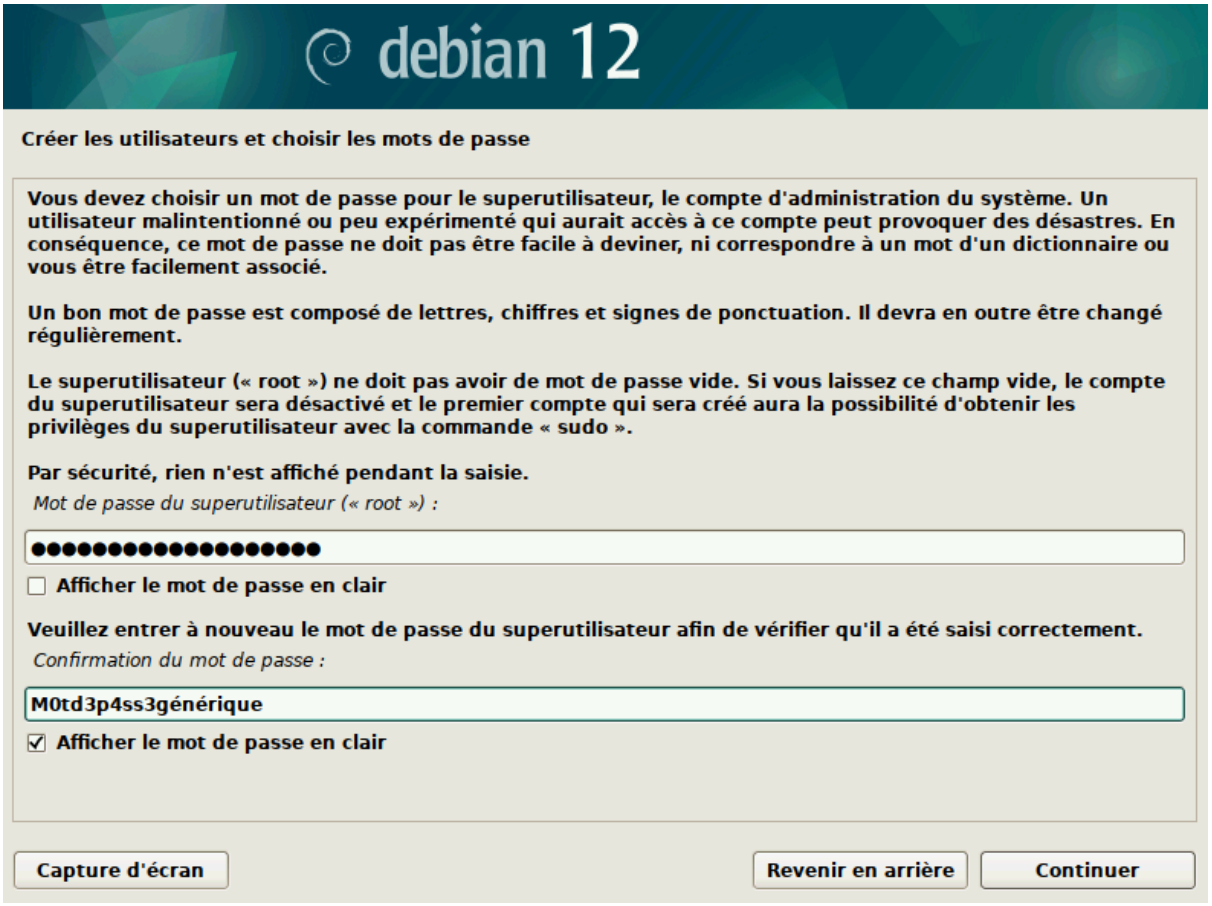
Ducoup j'ai fait ça là (j'ai pas de screen oops), et il faut changer les droits du fichiers de façon à ce que seul

Partie 2

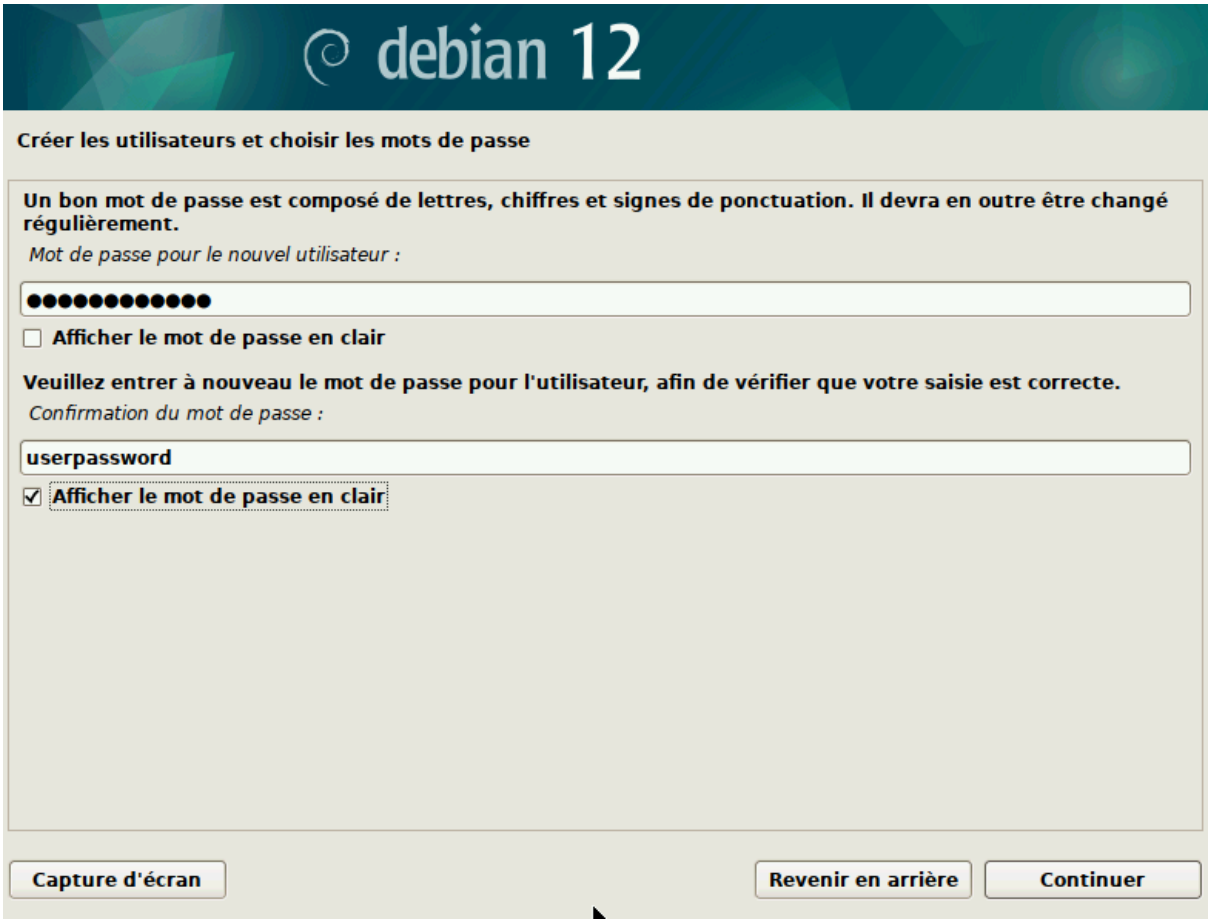




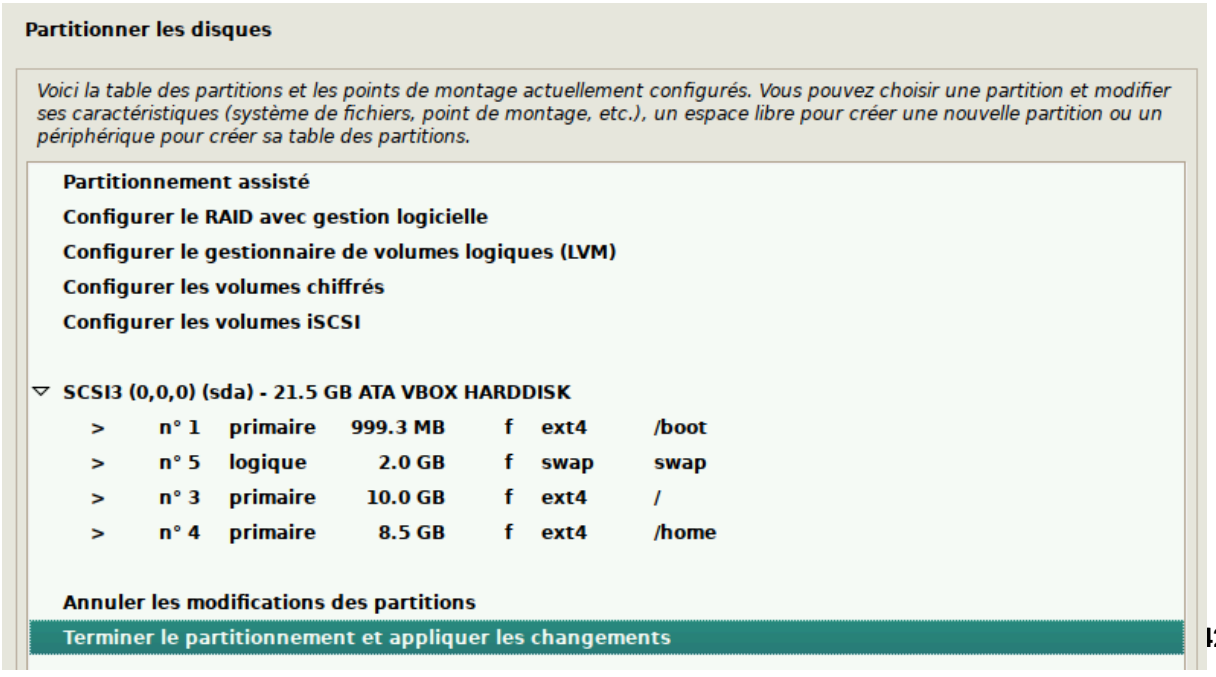
Je configure ma machine Debian, et je choisis ce mot de passe pour root.
Bien qu'il soit quelque peu commun et compréhensible par l'homme, il mettrait beaucoup de temps à être craqué (quoi que, tout dépends de la technique de brute force utilisé).

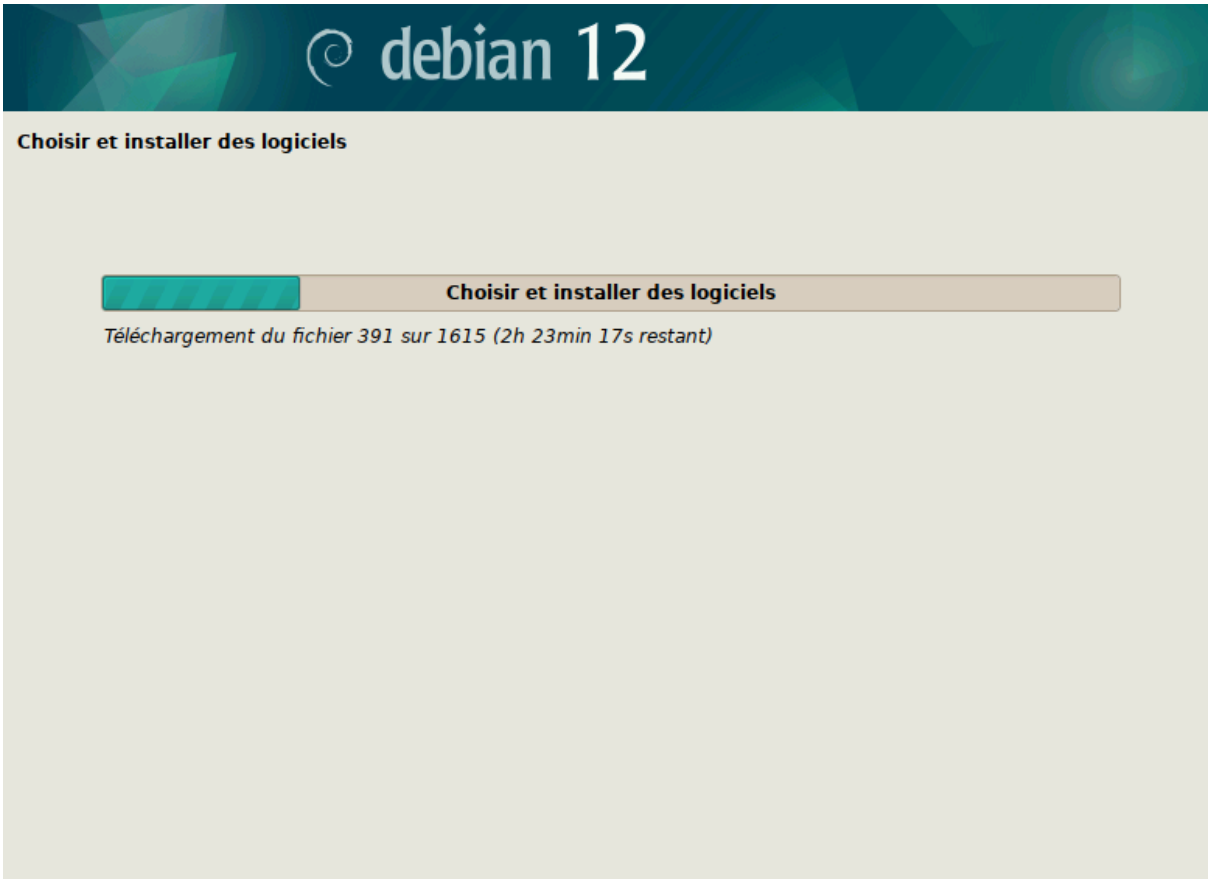


Je configure aussi l'utilisateur normal, dont le mot de passe est plus simple mais je ne pense pas qu'il soit nécéssaire que le mot de passe utilisateur soit bien compliqué puisque c'est par convention celui qui devrait le plus être utilisé (pas là vu que je vais devoir partitionner et chiffrer les données de ma machine).



Je partitionne maintenant le disque (manuellement) afin d'obtenir un debian prêt, avec une partition/le répertoire home prêt à être chiffré.





L'installation est très longue (voir interminable).

Mais ma machine de test debian toute configurée manuellement est maintenant prête à être utilisée :)

J'ai installé pCloud sur ma machine et je vais chiffrer mes données à l'intérieur.

```
utilisateur@VboxDebian:~/pCloudDrive$ mkdir MonChiffrement
utilisateur@VboxDebian:~/pCloudDrive$ ls -l
total 1
drwxr-xr-x 2 utilisateur utilisateur 4096 26 nov.  16:34 'Mes données'
drwxr-xr-x 2 utilisateur utilisateur 4096 27 nov.  19:48  MonChiffrement
utilisateur@VboxDebian:~/pCloudDrive$ go
gocryptfs       gocryptfs-xray  gold
utilisateur@VboxDebian:~/pCloudDrive$ gocryptfs
.directory       Mes données/     MonChiffrement/  .pcloudicon.png  .Trash-1000/
utilisateur@VboxDebian:~/pCloudDrive$ gocryptfs
.directory       Mes données/     MonChiffrement/  .pcloudicon.png  .Trash-1000/
utilisateur@VboxDebian:~/pCloudDrive$ gocryptfs -init MonChiffrement/
Choose a password for protecting your files.
Password:
Repeat:

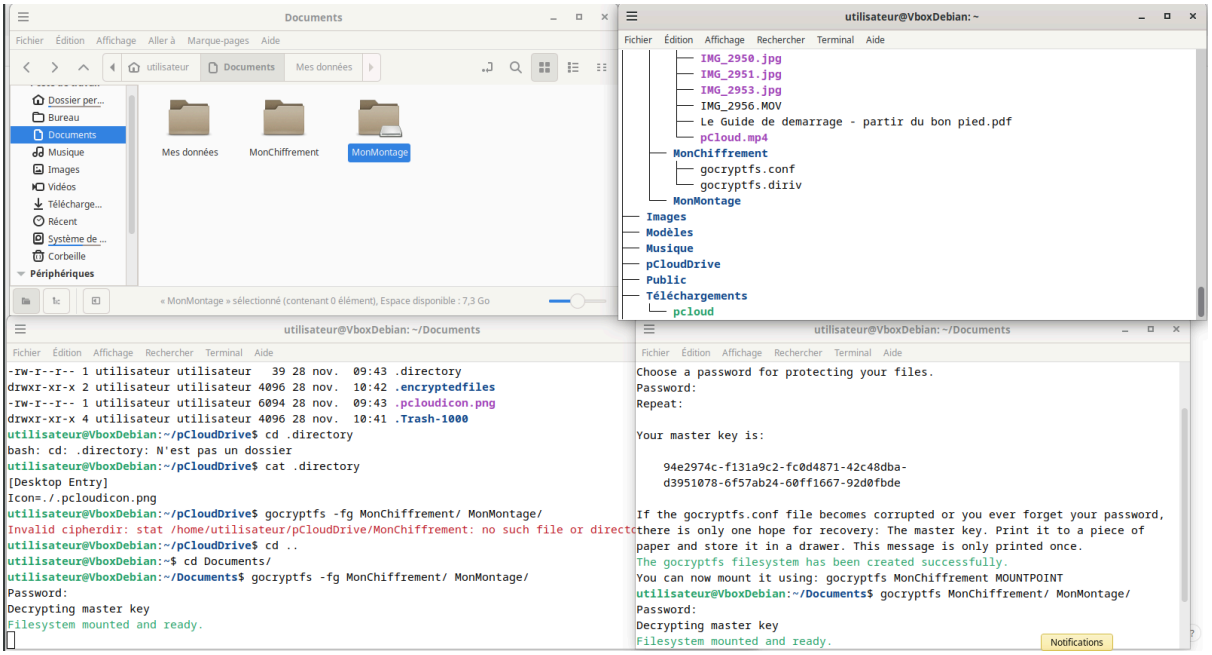
Your master key is:

    d15b8e7c-c7c0b32b-050617db-a0093ca5-
    14cf3471-b3e076d0-12978072-947781c6

If the gocryptfs.conf file becomes corrupted or you ever forget your password,
there is only one hope for recovery: The master key. Print it to a piece of
paper and store it in a drawer. This message is only printed once.
The gocryptfs filesystem has been created successfully.
You can now mount it using: gocryptfs MonChiffrement MOUNTPOINT
utilisateur@VboxDebian:~/pCloudDrive$
```

Je définis le mot de passe : "/:oipmlkjuyh" puisqu'il est dur à trouver mais facile à tapper (donc à retenir car mémoire musculaire) /:oipmlkjuyh

Maintenant je vais devoir configurer mon fichier de montage, où je placerais mes fichiers à chiffrer.



Je n'arrivais pas à monter mon dossier car il se trouvait dans mon pCloud. Donc, j'ai l'ai monté dans mon dossier Document, et ça marche.

Je vais donc mettre les fichiers chiffrés sur mon pCloud et les fichiers en clair sur ma machine Debian.

```
utilisateur@VboxDebian:~$ gocryptfs ./pCloudDrive/.encryptedfiles/ ./Documents/MonMontage/
Password:
Decrypting master key
Filesystem mounted and ready.
utilisateur@VboxDebian:~$ cp Documents/Mes\ données/* Documents/MonMontage/
utilisateur@VboxDebian:~$ fusermount -u Documents/MonMontage/
utilisateur@VboxDebian:~$
```

Les commandes à faire pour monter le répertoire de montage grâce à grocryptfs, puis pour les démonter (on voit que j'ai copier mes données